École Nationale des Sciences Appliquées

Marrakech

Rapport d'Analyse Statique APK

UnCrackable-Level1 (OWASP MSTG)

Lab 4 — Analyse Statique d'une Application Android

Réalisé par : Asma Bensassi Nour

Encadré par : Mohammed Lachgar

Filière : Cybersécurité

Année universitaire : 2025–2026

Avril 2026

Informations générales

Champ	Valeur
Fichier analysé	`UnCrackable-Level1.apk`
Provenance	OWASP MSTG Crackme — `https://mas.owasp.org/crackmes/Android/`
SHA-256	`1da8bf57d266109f9a07c01bf7111a1975ce01f190b9d914bcd3ae3dbef96f21`
Format vérifié	Archive ZIP valide (signature `PK` = `50 4B`)
Package	`owasp.mstg.uncrackable1`
Version	1.0 (versionCode : 1)
Min SDK / Target SDK	API 19 (Android 4.4) / API 28 (Android 9)
Environnement	Ubuntu Server minimal
Outils utilisés	
JADX	v1.5.0 — décompilation APK, analyse manifeste et code source
dex2jar	v2.4 — conversion `classes.dex` → `app.jar`
JD-GUI	v1.6.6 — visualisation du JAR pour analyse croisée
OpenJDK	v17 — runtime Java pour tous les outils

Résumé exécutif

L'application `UnCrackable-Level1` est un challenge pédagogique OWASP MSTG. Son analyse statique a révélé **6 vulnérabilités**, dont **2 de sévérité critique** compromettant directement la confidentialité du secret protégé.

#	Sévérité	Constat
1	Critique	Secret stocké en clair dans le code source
2	Critique	Vérification du secret effectuée côté client
3	Élevée	Protection root insuffisante et contournable
4	Élevée	Détection du mode debug exploitable
5	Moyenne	Sauvegarde ADB activée (`allowBackup=true`)
6	Faible	SDK minimal trop bas (API 19)

Actions prioritaires recommandées :

1. Supprimer tout secret en clair et migrer la validation vers un backend sécurisé.
2. Renforcer la détection root via la **Play Integrity API**.
3. Désactiver le mode debug en production et définir `allowBackup=false`.

Préparation de l'environnement et vérification de l'APK

L'environnement d'analyse a été configuré sur une machine Ubuntu Server. La vérification de l'intégrité de l'APK a été effectuée via les commandes suivantes :

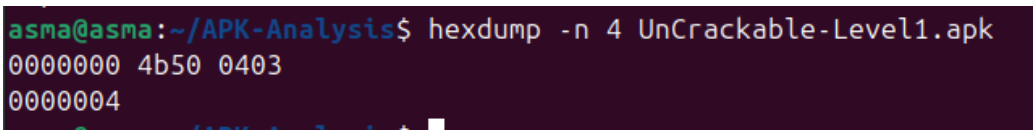

Figure 1 – Vérification de la signature ZIP de l'APK (hexdump)

```
asma@asma:~/APK-Analysis$ unzip -l UnCrackable-Level1.apk | head
Archive:  UnCrackable-Level1.apk
  Length      Date    Time    Name
---------  ---------- -----   ----
     1648  1980-00-00 00:00   AndroidManifest.xml
     1319  1980-00-00 00:00   META-INF/CERT.RSA
     1139  1980-00-00 00:00   META-INF/CERT.SF
     1077  1980-00-00 00:00   META-INF/MANIFEST.MF
     5528  1980-00-00 00:00   classes.dex
     1260  1980-00-00 00:00   res/layout/activity_main.xml
      456  1980-00-00 00:00   res/menu/menu_main.xml
asma@asma:~/APK-Analysis$ sha256sum UnCrackable-Level1.apk
1da8bf57d266109f9a07c01bf7111a1975ce01f190b9d914bcd3ae3dbef96f21  UnCrackable-Level1.
apk
```

FIGURE 2 – Analyse de la structure interne de l'APK avec unzip ET Calcul de l'empreinte SHA-256 de l'APK

La signature `50 4B` confirme que le fichier est une archive ZIP valide, conforme au format APK Android standard.

Contenu identifié dans l'APK :

```
asma@asma:~/APK-Analysis$ unzip -l UnCrackable-Level1.apk
Archive:  UnCrackable-Level1.apk
  Length      Date    Time    Name
---------  ---------- -----   ----
     1648  1980-00-00 00:00   AndroidManifest.xml
     1319  1980-00-00 00:00   META-INF/CERT.RSA
     1139  1980-00-00 00:00   META-INF/CERT.SF
     1077  1980-00-00 00:00   META-INF/MANIFEST.MF
     5528  1980-00-00 00:00   classes.dex
     1260  1980-00-00 00:00   res/layout/activity_main.xml
      456  1980-00-00 00:00   res/menu/menu_main.xml
     4751  1980-00-00 00:00   res/mipmap-hdpi-v4/ic_launcher.png
     2348  1980-00-00 00:00   res/mipmap-mdpi-v4/ic_launcher.png
     7275  1980-00-00 00:00   res/mipmap-xhdpi-v4/ic_launcher.png
    14398  1980-00-00 00:00   res/mipmap-xxhdpi-v4/ic_launcher.png
    22801  1980-00-00 00:00   res/mipmap-xxxhdpi-v4/ic_launcher.png
     2748  1980-00-00 00:00   resources.arsc
---------                     -------
```

FIGURE 3 – Analyse de la structure interne de l'APK avec la commande unzip

Fichier	Rôle
`AndroidManifest.xml`	Configuration principale de l'application
`classes.dex`	Bytecode Dalvik de l'application
`META-INF/CERT.RSA`	Certificat de signature
`META-INF/CERT.SF`	Fichier de signature
`META-INF/MANIFEST.MF`	Manifeste de l'archive
`res/layout/activity_main.xml`	Interface utilisateur principale
`res/menu/menu_main.xml`	Menu de l'application

Conversion DEX → JAR avec dex2jar

Objectif

La conversion du fichier `classes.dex` (bytecode Dalvik) en fichier `app.jar` (bytecode Java standard) permet une analyse alternative via JD-GUI et offre une vue croisée complémentaire à JADX.

Procédure exécutée

- Extraction du fichier `classes.dex` depuis l'APK.
- Conversion du fichier DEX en fichier JAR à l'aide de l'outil `dex2jar`.
- Vérification de la génération du fichier `app.jar`.
- Analyse du fichier JAR avec l'outil JD-GUI.

```
^Casma@asma:~/bin$ ls ~/APK-Analysis
UnCrackable-Level1.apk
asma@asma:~/bin$ cd ~/APK-Analysis
asma@asma:~/APK-Analysis$ mkdir dex_out
asma@asma:~/APK-Analysis$ unzip -j UnCrackable-Level1.apk "classes*.dex" -d dex_out
Archive:  UnCrackable-Level1.apk
  inflating: dex_out/classes.dex
asma@asma:~/APK-Analysis$ ls dex_out
classes.dex
asma@asma:~/APK-Analysis$ cd ~/dex-tools-v2.4
asma@asma:~/dex-tools-v2.4$ ./d2j-dex2jar.sh ~/APK-Analysis/dex_out/classes.dex -o ~/AP
K-Analysis/app.jar
dex2jar /home/asma/APK-Analysis/dex_out/classes.dex -> /home/asma/APK-Analysis/app.jar
asma@asma:~/dex-tools-v2.4$ ls ~/APK-Analysis
UnCrackable-Level1.apk  app.jar  dex_out
```

FIGURE 4 – Extraction du fichier DEX, conversion en JAR

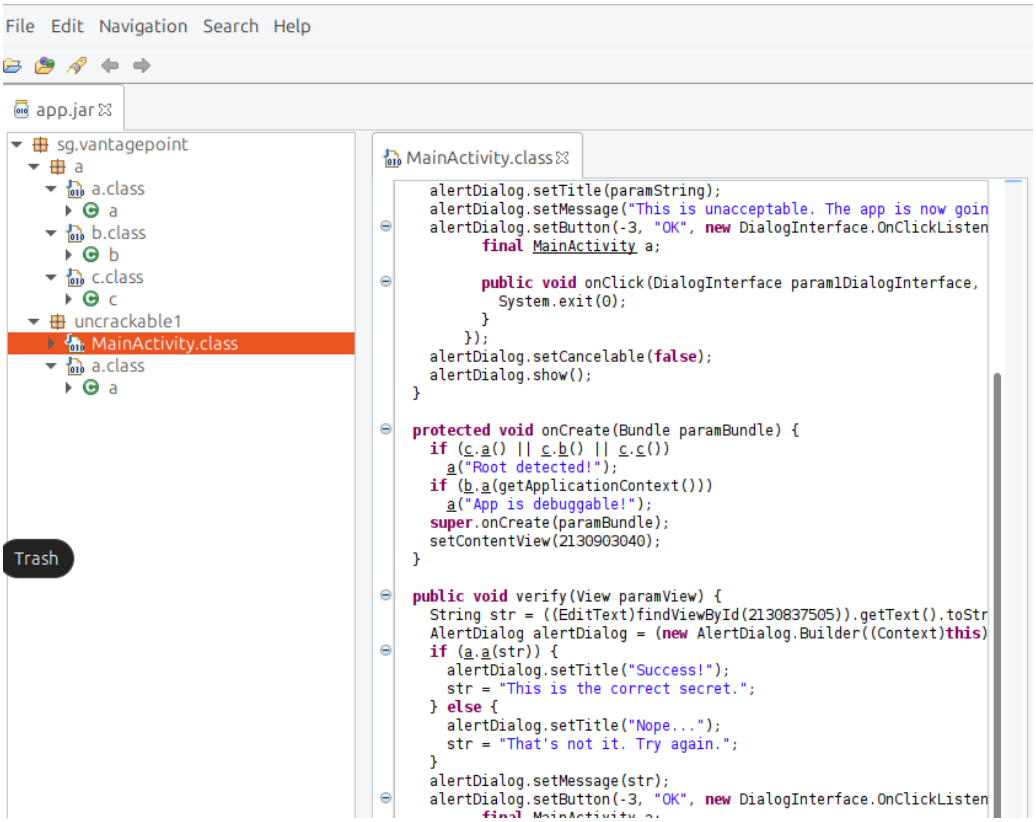

FIGURE 5 – analyse avec JD-GUI avec la commande java -jar /jd-gui-1.6.6.jar

Observations

La conversion s'est effectuée sans erreur. Le fichier `app.jar` généré contient l'ensemble des classes Java décompilées, notamment :

- `sg.vantagepoint.uncrackable1.MainActivity` — classe principale avec la logique de vérification
- `sg.vantagepoint.a.a` — classe contenant la méthode de validation cryptographique

— `sg.vantagepoint.a.b` — classe de détection du mode debug
— `sg.vantagepoint.a.c` — classe de détection root (méthodes `a()`, `b()`, `c()`)

Comparaison JADX vs JD-GUI

Méthodologie

La même classe `sg.vantagepoint.uncrackable1.MainActivity` a été analysée dans les deux outils afin d'établir une comparaison objective.

Tableau comparatif

Aspect	JADX v1.5.0	JD-GUI v1.6.6
Source d'entrée	Ouvre l'APK directement	Nécessite un JAR préalablement généré par dex2jar
Accès aux ressources	Accès complet : manifeste, `strings.xml`, layouts, assets	Aucun accès aux ressources Android (XML, manifeste)
Lisibilité du code	Code Java reconstruit proprement avec annotations et types Android préservés	Code fonctionnel mais parfois moins lisible ; certaines constructions Android mal traduites
Recherche globale	Recherche textuelle (Ctrl+F) dans tout le projet : code, manifeste, ressources	Pas de recherche globale multi-classes
Gestion obfuscation	Tente de reconstruire les noms de variables et de méthodes	Conserve les noms obfusqués tels quels
Cas d'usage idéal	Analyse complète et approfondie d'un APK inconnu	Validation croisée rapide du code Java

Conclusion

JADX s'est révélé l'outil le plus complet pour cette analyse grâce à sa recherche globale et à l'accès intégré au manifeste. JD-GUI constitue un complément utile pour vérifier le code Java indépendamment de la chaîne de décompilation de JADX. Dans un contexte professionnel, l'utilisation combinée des deux outils est recommandée pour une couverture maximale.

Constats détaillés

Constat #1 — Secret exposé en clair

Sévérité : Critique

Localisation : `sg.vantagepoint.uncrackable1.MainActivity.verify(View)`

Description : La chaîne `"This is the correct secret."` est stockée directement dans le code source de la méthode `verify()`. Elle est visible immédiatement lors d'une recherche sur le terme *secret* dans JADX.

Impact : Un attaquant peut récupérer ce secret en moins d'une minute via tout outil de reverse engineering, sans exécuter l'application.

Remédiation :

— Ne jamais stocker de secret en clair dans le code source ou les ressources.
— Utiliser le **Android Keystore** pour le stockage sécurisé des secrets.
— Déplacer la logique de validation vers un **backend sécurisé** via une API authentifiée.

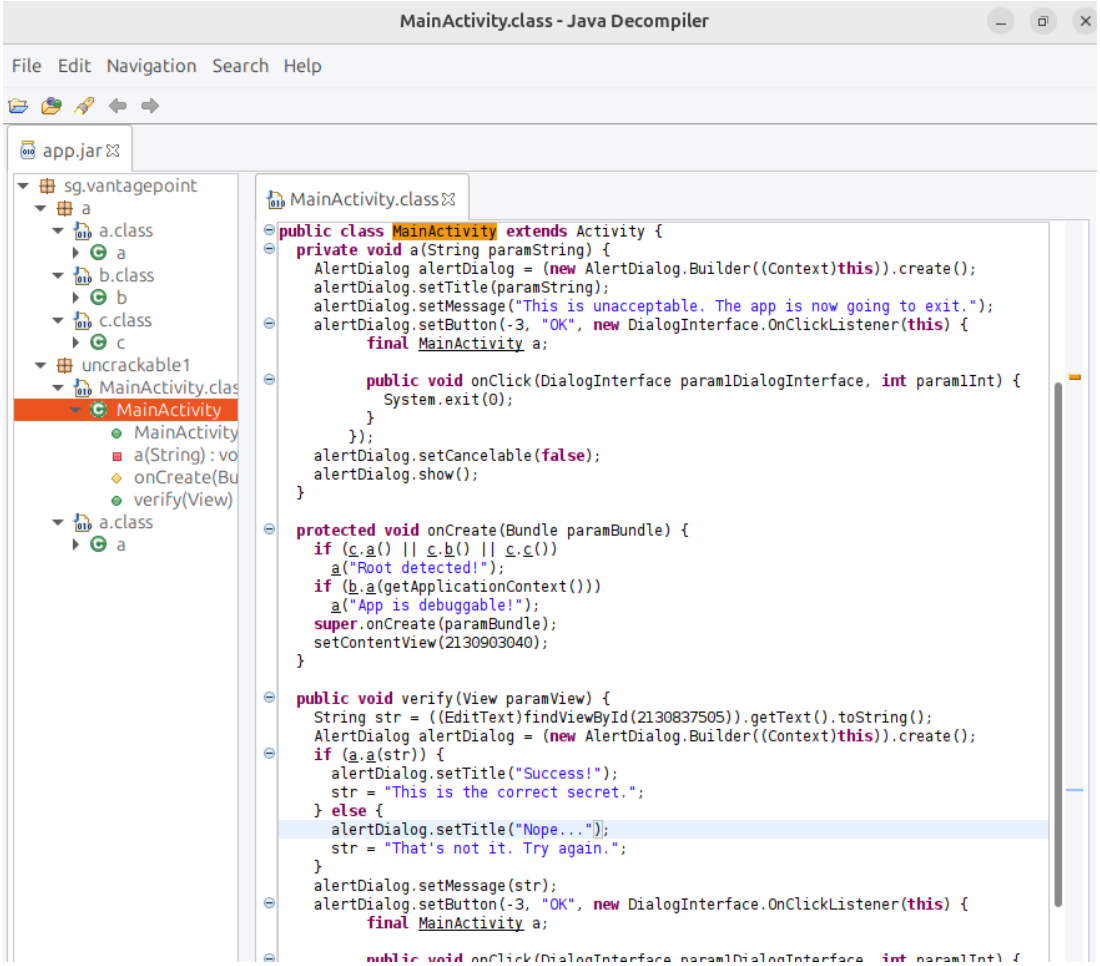

FIGURE 6 – Présence du secret en clair et logique de validation côté client dans la méthode verify de l'application

Constat #2 — Vérification du secret côté client

Sévérité : Critique

Localisation : `sg.vantagepoint.uncrackable1.MainActivity.verify(View) → appel a.a(str)`

Description : La validation du secret est effectuée intégralement dans l'application via la méthode `a.a(str)`. La logique de comparaison est exposée dans le bytecode et peut être analysée sans exécuter l'application.

Impact : Un attaquant peut identifier la condition de succès et contourner la vérification par patching de l'APK ou par hooking dynamique (ex. Frida).

Remédiation :

— Effectuer toute validation sensible exclusivement côté serveur.
— Ne jamais implémenter de logique de sécurité critique uniquement dans le client mobile.

Constat #3 — Protection root insuffisante

Sévérité : Élevée

Localisation : `sg.vantagepoint.uncrackable1.MainActivity.onCreate(Bundle) → c.a() || c.b() || c.c()`

Description : L'application détecte un environnement rooté via trois méthodes dans la classe

`sg.vantagepoint.a.c` : vérification de la chaîne `"test-keys"`, présence de fichiers système spécifiques (`su`, `busybox`), et vérification de chemins système. En cas de détection, l'application se ferme brutalement via `System.exit(0)`.

Impact : Ces méthodes sont connues et documentées ; elles peuvent être contournées par modification de l'environnement, patching de l'APK ou hooking. La fermeture brutale constitue en outre un risque de déni de service.

Remédiation :

— Remplacer par la **Play Integrity API** pour une détection fiable côté serveur.
— Adopter une approche multi-niveaux (détection native + contrôles serveur).
— Remplacer `System.exit(0)` par une dégradation gracieuse du service.

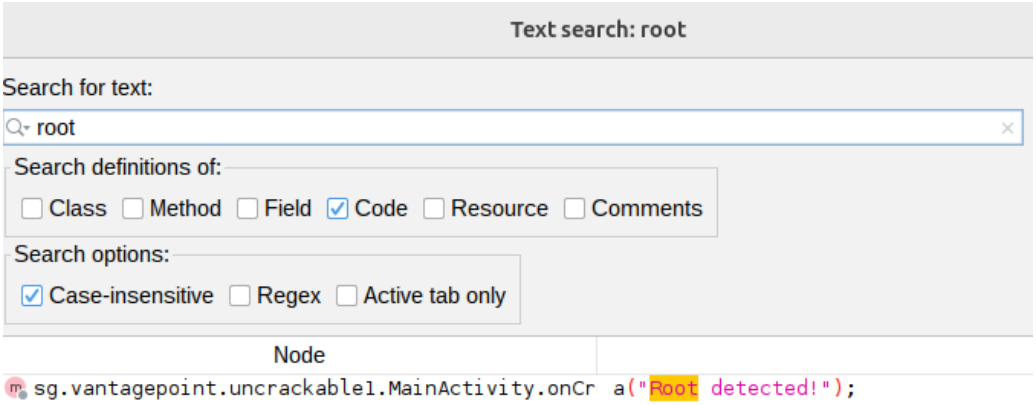

FIGURE 7 – Détection d'un environnement rooté dans la méthode onCreate

Constat #4 — Application débogable

Sévérité : Élevée

Localisation : `sg.vantagepoint.uncrackable1.MainActivity.onCreate(Bundle) → b.a(getApplicationContext(`

Description : L'application vérifie le mode debug via la méthode `b.a()` de la classe `sg.vantagepoint.a.b` et affiche le message `"App is debuggable!"`. La détection est observable statiquement via la recherche du terme *debug* dans JADX.

Impact : Un attaquant peut utiliser des outils de débogage (JDWP, Frida, adb) pour intercepter et modifier le comportement de l'application à l'exécution.

Preuve

Remédiation :

— Définir `android:debuggable="false"` dans le manifeste pour les builds de production.
— Intégrer des protections anti-debugging dynamiques dans le code natif.

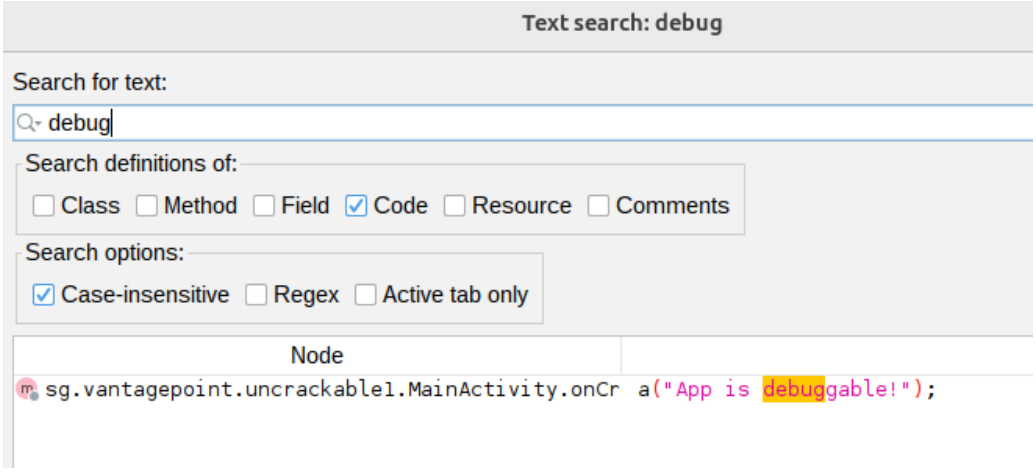

FIGURE 8 – Détection du mode debug dans l'application

Constat #5 — Sauvegarde ADB activée

Sévérité : Moyenne

Localisation : `AndroidManifest.xml → balise <application android:allowBackup="true">`

Description : L'attribut `android:allowBackup="true"` est défini dans le manifeste, autorisant la sauvegarde des données de l'application via Android Debug Bridge (ADB).

Impact : Un attaquant disposant d'un accès physique ou USB à l'appareil peut extraire les données locales de l'application (préférences partagées, bases de données SQLite) avec la commande `adb backup`.

```
        android:targetSdkVersion="28"/>
11  <application
        android:theme="@style/AppTheme"
        android:label="@string/app_name"
        android:icon="@mipmap/ic_launcher"
        android:allowBackup="true">
16      <activity
            android:label="@string/app_name"
```

FIGURE 9 – Configuration allowBackup activée dans le manifeste

Remédiation : Définir `android:allowBackup="false"` dans `AndroidManifest.xml`.

Constat #6 — Version SDK minimale trop basse

Sévérité : Faible

Localisation : `AndroidManifest.xml → <uses-sdk android:minSdkVersion="19">`

Description : L'application supporte Android 4.4 (API 19), une version ne bénéficiant plus des correctifs de sécurité Android modernes ni des fonctionnalités de sécurité récentes (ex. `StrictMode` renforcé, protections mémoire).

Impact : Exposition potentielle à des vulnérabilités système non corrigées sur les appareils anciens.

AndroidManifest.xml × MainActivity ×

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="1"
    android:versionName="1.0"
    package="owasp.mstg.uncrackable1">
    <uses-sdk
        android:minSdkVersion="19"
        android:targetSdkVersion="28"/>
    <application
        android:theme="@style/AppTheme"
        android:label="@string/app_name"
        android:icon="@mipmap/ic_launcher"
        android:allowBackup="true">
        <activity
            android:label="@string/app_name"
            android:name="sg.vantagepoint.uncrackable1.MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>
</manifest>
```

FIGURE 10 – Configuration des versions SDK dans le fichier AndroidManifest.xml

Remédiation : Augmenter `minSdkVersion` à **26** minimum (Android 8.0), conformément aux recommandations OWASP MASVS L1.

Annexes

A. Permissions déclarées

Permission	Risque
`android.permission.INTERNET`	Faible — permet l'accès réseau

Observation : L'application ne demande qu'une seule permission, ce qui est cohérent avec sa fonction de challenge local. Aucune permission excessive n'a été identifiée.

B. Composants déclarés dans le manifeste

Composant	Type	Exporté
`sg.vantagepoint.uncrackable1.MainActivity`	Activity	Oui (intent-filter MAIN/LAUNCHER)

C. Extraits de code significatifs

Méthode verify() — Secret en clair et vérification côté client :

```java
public void verify(View view) {
    String str;
    String obj = ((EditText) findViewById(R.id.edit_text)).getText().toSt
    AlertDialog create = new AlertDialog.Builder(this).create();
    if (a.a(obj)) {
        create.setTitle("Success!");
        str = "This is the correct secret.";
    } else {
        create.setTitle("Nope...");
        str = "That's not it. Try again.";
    }
```

FIGURE 11 – Présence du secret en clair dans le code source

Méthode onCreate() — Détection root et debug :

```java
@Override // android.app.Activity
protected void onCreate(Bundle bundle) {
    if (c.a() || c.b() || c.c()) {
        a("Root detected!");
    }
    if (b.a(getApplicationContext())) {
        a("App is debuggable!");
    }
    super.onCreate(bundle);
    setContentView(R.layout.activity_main);
}
```

FIGURE 12 – Détection d'un environnement rooté

D. Références

— OWASP MSTG — `https://owasp.org/www-project-mobile-security-testing-guide/`
— OWASP MASVS — `https://mas.owasp.org/MASVS/`
— Android Developers — Sécurité : `https://developer.android.com/topic/security`
— Play Integrity API : `https://developer.android.com/google/play/integrity`
— JADX : `https://github.com/skylot/jadx`
— dex2jar : `https://github.com/pxb1988/dex2jar`

Rapport généré dans le cadre d'un projet académique de cybersécurité mobile.
Toutes les analyses ont été effectuées sur un APK autorisé à des fins strictement pédagogiques.